MODULE NAME:	MODULE CODE:
PROGRAMMING 2A	PROG6221/w

ASSESSMENT TYPE: POE (PAPER)

TOTAL MARK ALLOCATION: 300 MARKS

TOTAL HOURS: A minimum of 35 HOURS is suggested to complete this assessment

By submitting this assignment, you acknowledge that you have read and understood all the rules as per the terms in the registration contract, in particular the assignment and assessment rules in The IIE Assessment Strategy and Policy (IIE009), the intellectual integrity and plagiarism rules in the Intellectual Integrity and Property Rights Policy (IIE023), as well as any rules and regulations published in the student portal.

INSTRUCTIONS:

1. ***No material may be copied from original sources, even if referenced correctly, unless it is a direct quote indicated with quotation marks. No more than 10% of the assignment may consist of direct quotes.***
2. *Assignments must be typed unless otherwise specified.*
3. *Begin each section on a new page.*
4. *Follow all instructions on the PoE cover sheet.*
5. *This is an individual assignment.*

Referencing Rubric

Providing evidence based on valid and referenced academic sources is a fundamental educational principle and the cornerstone of high-quality academic work. Part of achieving this quality is referencing in a way that is consistent and congruent with the requirements of the referencing style being used.

Therefore, inconsistent and/or incongruent referencing will result in a penalty of **a maximum of ten percent being deducted from the overall percentage** awarded to your assessment submission.

Please note that **evidence of plagiarism** in the form of copied or unreferenced work, absent reference lists, or exceptionally poor referencing **may result in action being taken in accordance with The IIE's Intellectual Integrity and Property Rights Policy (IIE023)**. Similarly, **evidence of excessive AI usage may result in action being taken in accordance with The IIE's Student Conduct, Discipline and Safety Policy (IIE015)**.

Markers are required to provide feedback to students by **circling/underlining the information in the table below that best describes the student's work and by adding constructive commentary where appropriate.** The examples provided are not exhaustive but illustrate the errors.

Deductions

- Where the student's work contains **five or more errors** aligned to the **minor errors column** below, **deduct 5% from the overall percentage.**
- Where the student's work contains **five or more errors** aligned to the **major errors column** below, **deduct 10% from the overall percentage.**
- **Where both minor and major errors** (e.g. two minor and three major, etc.) are present, **deduct 10% only** (and not 5% or 15%) from the overall percentage.

Required: **Consistent and congruent referencing**	**Minor errors** **Deduct 5% from overall percentage.** **Example: if the response receives 70%, deduct 5%. The final mark is 65%.**	**Major errors** **Deduct 10% from the overall percentage.** **Example: if the response receives 70%, deduct 10%. The final mark is 60%.**
Consistency • The **correct referencing style** for the **discipline** – i.e., either **Harvard**, OR **APA** (for Psychology)**,** OR **Law**, OR **IEEE** (for ICT/Engineering) – has been used consistently for all in-text references and in the bibliography/reference list. • **Concepts** and **ideas** that are **quoted** and/or **paraphrased** are referenced consistently throughout. • **Position of the in-text reference**: an in-text reference is positioned consistently where appropriate for every quote and paraphrase.	**Minor inconsistencies:** • The referencing style used is generally consistent with what is required, but there are one or two changes/errors in the format of in-text referencing and/or in the bibliography/reference list. • For example, page numbers for direct quotes in-text have been provided for one source, but not in another. Or, two book chapters in the bibliography/reference list have been referenced in two different formats. Or, the publication year has been placed after the author name in one bibliography/reference list entry, and after the source title in another, etc. • Concepts and ideas in quotes and/or paraphrases are typically referenced, but a full in-text reference is missing or incomplete from one or two small sections of the work. • Position of the references: in-text references are only given at the beginning and/or end of every paragraph.	**Major inconsistencies:** • Poor and wholly inconsistent referencing style used in-text and/or in the bibliography/reference list. • Multiple referencing styles for the same source types have been used. • For example, the format for direct quotes in-text and/or book chapters in the bibliography/reference list and/or year of publication in the bibliography/reference list is different across multiple instances. • Concepts and ideas in quotes and/or paraphrases are haphazardly referenced in-text. • Position of the references: in-text references are only given at the beginning or end of large sections of work.
Feedback on referencing consistency:		
Congruency • **Each source** reflected within **in-text references** is included accurately in the **bibliography/reference list.** • All **bibliography/reference list** entries are in the **required order** for the referencing style used (e.g. alphabetical, alphabetical under subheadings, numerical). • All **direct quotes** and **paraphrases** have been **integrated appropriately** into the text using introductory phrases, accurate grammar, etc.	**Minor incongruences:** • There is largely a match between the sources presented in-text and those in the bibliography/reference list, but one or two sources that appear in-text do not appear in the bibliography/reference list, or vice versa. Or key source information is missing from one or two in-text references or bibliography/reference list entries only (e.g. publication year, city of publication, URL date accessed, etc.). • There is a clear and largely accurate ordering of sources in the bibliography/reference list as required by the referencing style used, but with one or two references out of order. • An attempt has been made for source integration into the text using appropriate introductory phrases and grammar, but one or two quotes or paraphrases do not flow as clearly or logically within the sentence structure as they could.	**Major incongruences:** • No relationship/several incongruencies between the in-text referencing and the bibliography/reference list. • For example, multiple sources are included in-text, but not in the bibliography, and/or vice versa. Key source information is missing from multiple in-text references and/or reference list entries. A URL link, rather than the actual reference, is provided in the bibliography. Sources are repeated in the reference list, etc. • Most sources are listed in a haphazard order throughout the bibliography/reference list. • Few to no appropriate introductory phrases or rules of grammar have been applied, and many direct quotes and/or paraphrases feel disconnected from the flow of the text.
Feedback on referencing congruency:		
Overall feedback on referencing, with suggested improvements:		

Background

Cybersecurity Awareness Chatbot for South African Citizens

In recent years, South Africa has seen a significant rise in cyberattacks targeting individuals, businesses, and government institutions (Pieterse, 2021). These attacks often include phishing scams, malware, and social engineering, leaving many people vulnerable to financial loss, identity theft, and psychological harm. In response to these growing threats, the Department of Cybersecurity has launched a campaign aimed at educating citizens on identifying and mitigating cyber threats. To support this campaign, you have been tasked with developing a chatbot, a virtual assistant that interacts with users to educate them on cybersecurity topics in a conversational manner.

This chatbot will serve as a "Cybersecurity Awareness Assistant." Its purpose is to simulate real-life scenarios where users might encounter cyber threats and provide guidance on avoiding common traps. The bot will cover topics like phishing emails, safe password practices, and recognising suspicious links. By interacting with the chatbot, users will gain practical knowledge about cybersecurity while you develop valuable programming skills. As you progress through each part of this project, you will expand the chatbot's capabilities, transforming it from a simple conversation bot into a sophisticated tool for user engagement and education.

With cybersecurity awareness, which is now a national priority in South Africa, this project presents a meaningful way to educate users while honing your programming abilities. Embrace this opportunity to create a tool that could make a genuine impact on cybersecurity literacy!

Further reading

Pieterse, H. 2021. The Cyber Threat Landscape in South Africa: A 10-Year Review. *The African Journal of Information and Communication*, 28(28). doi: https://doi.org/10.23962/10539/32213. [Online]. Available at: https://www.scielo.org.za/scielo.php?pid=S2077-72132021000200003&script=sci_arttext [Accessed 16 February 2026].

Introduction

This Portfolio of Evidence (POE) consists of three parts, each of which builds on the previous one. You will submit two parts during the semester, with a final submission at the end of the semester. Each part of the project must be preserved carefully, as later parts will depend on the work you complete in earlier parts.

- **Part 1**: Develop a command-line application that functions as a basic cybersecurity awareness chatbot, allowing users to engage in a simulated conversation about online safety practices.
- **Part 2**: Update the command-line application to now have a Graphic User Interface using Windows Presentation Foundation (WPF) OR Windows Forms (Win Forms). The bot should be able to recognise specific topics related to cybersecurity.
- **Part 3/POE**: Extend the existing GUI from task 2 and add advanced features, including interactive elements like a simple game or task list for cybersecurity tips. Create a professional and engaging 8 to 10-minute video presentation (see rubric).

Note on Real-World Development: Software projects often experience unexpected requirement changes, which reflect real-world development scenarios. However, you have the benefit of knowing all three parts of the project requirements in advance, so review all parts before starting Part 1. This will help you plan effectively and avoid unnecessary rework.

Instructions

Important Notes

- Submission: Each part must be submitted correctly pushed to GitHub with the link submitted on Arc for each part, Part 1, Part 2 and POE.
- GitHub Requirements: Each part of the project should have a **minimum of six commits**, with Continuous Integration (CI) implemented through GitHub Actions and Workflow. You will lose 5% per task if you do not hand in on GitHub.
- Presentation Requirement: You are required to produce a video presentation of your code for part 1, part 2 and the final POE. This should include a full explanation and demonstration of your application running. Please note that you must also explain how your logic and flow are produced. This is worth marks, so do not leave it out. Please provide an unlisted YouTube video or a platform like that, which needs to include a voice-over of your own voice and no AI voices to be used.

Marks are awarded for **functional**, **running software**, **not merely source code**. Ensure your project compiles and your README file provides adequate instructions for running the program.

Marks will only be given for working code.

Part 1 — Basic Chatbot Interaction with Voice Greeting, Image (Marks: 100)

Learning Units 1 and 2.

At the end of this specific part, students should be able to:

- Write a console programme that requires user input.
- Apply string manipulation to solve a programming problem.
- Use automatic properties to solve a programming problem.

Description

In Part 1, you will develop the foundational chatbot structure, providing a friendly greeting, user interaction, predefined responses, input validation, and using GitHub for project version control and CI workflows. Additionally, you will enhance the chatbot's console interface with multimedia elements such as a voice greeting, an ASCII image, and colour formatting.

Questions

1. Voice Greeting

Objective: Add a personalised touch to the chatbot by playing a recorded voice greeting when the application launches.

Tasks

- Record a short voice message welcoming the user to the chatbot (e.g., "Hello! Welcome to the Cybersecurity Awareness Bot. I'm here to help you stay safe online.").
- Save the audio file in WAV format for compatibility with System.Media in C#.
- Play the audio file when the application starts.

2. Image Display

Objective: Enhance the chatbot's visual presentation by displaying an ASCII representation of a logo or relevant image.

Tasks

- Use ASCII art to create a "Cybersecurity Awareness Bot" logo or a cybersecurity-themed symbol.
- Display this image as a header or title screen when the chatbot launches.

3. Text-Based Greeting and User Interaction

Objective: Initiate a welcoming and interactive experience.

Tasks

- Ask the user for their name and personalise responses using the name.
- Display a text-based welcome message with the ASCII art or decorative borders after the voice greeting plays.

4. Basic Response System

Objective: Respond to basic cybersecurity-related user questions.

Tasks

- Create responses for questions like "How are you?", "What's your purpose?", and "What can I ask you about?"
- Include topics related to password safety, phishing, and safe browsing.

5. Input Validation

Objective: Handle unexpected or invalid inputs gracefully.

Tasks

- Detect and respond to invalid inputs, such as empty entries or unsupported queries.
- Provide a default response, like "I didn't quite understand that. Could you rephrase?"

6. Enhanced Console UI with Visual Elements

Objective: Improve the chatbot's appearance using formatting techniques.

Tasks

- Use coloured text, spacing, borders, and symbols to create a visually structured interface.
- Add section headers and dividers for readability and structure.
- Consider using a typing effect or slight delays to simulate a conversational feel.

7. Code Structure and Readability

- Ensure code is structured clearly.
- Prevent writing all the code in the Program.cs file.
- Make use of methods and classes for clear structure and readability.

8. GitHub Version Control and CI Implementation

Commit Your Code to GitHub

- Make a minimum of six meaningful commits in your GitHub repository. Each commit should reflect a significant change or addition to the project, with descriptive commit messages.

Example messages

- Initial commit: Set up project structure and main files.
- Added greeting and user interaction functionality.
- Implemented basic cybersecurity responses and input validation.

Set Up GitHub Actions for Continuous Integration (CI):

- Set up a CI workflow using GitHub Actions. This workflow should check for basic requirements such as syntax errors, code formatting, or a successful build.
- In the GitHub repository, navigate to the Actions tab and choose a preconfigured workflow or set up a custom one that checks your code with each push.
- Save the workflow file in the .github/workflows folder in your repository.

Ensure CI Workflow Functionality by the Submission Deadline:

Verify that your CI workflow is operational and completes successfully (green check mark) with each commit.

Submission Instructions

ARC Submission

- Submit ONLY your GitHub link on ARC, which includes the following:
  - Complete project folder, including source code, the README file, and all multimedia files (WAV file for the voice greeting and ASCII art image).
- Ensure a **screenshot** of a successful CI workflow run (green check mark) from GitHub Actions is in your README file.
- Submit your presentation for Task 1 as a **YouTube unlisted link**. Ensure the video is clearly recorded with a voice-over, explaining the code structure, logic, and techniques used for voice integration and formatting.

GitHub Submission

- Ensure your repository has a **minimum of six commits** with meaningful messages.
- Set up **GitHub Actions** for CI with automated checks.
- Include the **WAV file** and **ASCII art** in the repository, and ensure the final commit passes all CI checks.

Part 2 — GUI interface and Dynamic Responses, Sentiment Detection, and Memory (Marks: 100)

All Learning Units

At the end of this specific part, students should be able to:

- Create a GUI application using WPF or WinForm.
- Use a generic collection to solve a programming problem.
- Use delegates to solve a programming problem.

Description

In Part 2, you will expand the Cybersecurity Awareness Chatbot by designing a Graphical User Interface to make the system more engaging and user-friendly. The user should now use the designed GUI to interact with the bot. The responses should be more dynamic, recognise specific cybersecurity-related keywords, and incorporate memory to recall information shared by the user. Additionally, you will implement **sentiment detection**, enabling the chatbot to adjust responses based on the user's mood or tone, enabling the bot to be more interactive and simulate a natural conversation flow.

Questions

1. GUI design and implementation

- Translate ALL task 1 requirements into the GUI application, including the voice implementation.
- Ensure that the GUI is designed neatly, with user friendly design, use of colours and proper spacing of design elements.
- Ensure the ASCII art implemented in task 1 is translated into the GUI effectively.

2. Keyword Recognition

Objective: Enhance the chatbot's ability to recognise and respond to specific cybersecurity-related topics and general inquiries.

Tasks

- Implement keyword recognition so the chatbot can identify cybersecurity topics in user input and provide relevant responses.

- Recognise at least three keywords related to cybersecurity, such as "password," "scam," and "privacy," with responses that provide guidance on each.

Example

User: "Tell me about password safety."

Chatbot: (*Recognises "password" and responds with a cybersecurity tip*) "Make sure to use strong, unique passwords for each account. Avoid using personal details in your passwords."

3. Random Responses

Objective: Introduce variation by having the chatbot select from multiple predefined responses for common cybersecurity queries.

Tasks

- For some cybersecurity topics, such as "phishing tips", ensure the chatbot randomly selects one of several informative responses.
- Use **arrays** or **lists** to manage these responses effectively to keep interactions varied and engaging.

Example

User: "Give me a phishing tip."

Chatbot: *(Randomly selects from a list of responses*) "Be cautious of emails asking for personal information. Scammers often disguise themselves as trusted organisations."

4. Conversation Flow

Objective: Enable the chatbot to maintain a conversational flow that feels natural and responsive.

Tasks

- Ensure the chatbot can handle follow-up questions or statements without restarting the conversation, creating a seamless flow. E.g. *"Give me another tip", or "explain more", or "tell me more"*.
- If the user expresses confusion or asks for more details, the chatbot should continue the current topic, especially for important cybersecurity topics.

5. Memory and Recall

Objective: Implement a memory feature to allow the chatbot to "remember" user details and refer back to them.

Tasks

- Store certain information provided by the user (e.g., their name, favourite cybersecurity topic) and use it later in the conversation.
- Use this feature to personalise responses and make cybersecurity tips more engaging.

Example

User: "I'm interested in privacy."

Chatbot: "Great! I'll remember that you're interested in privacy. It's a crucial part of staying safe online."

Later in the conversation:

Chatbot: "As someone interested in privacy, you might want to review the security settings on your accounts."

6. Sentiment Detection

Objective: Allow the chatbot to adjust its responses based on the user's sentiment to create an empathetic interaction.

Tasks

- Detect simple sentiments such as "worried", "curious", or "frustrated" when discussing cybersecurity topics.
- Adjust the chatbot's responses accordingly to provide encouragement or support if the user feels overwhelmed or unsure.

Example

User: "I'm worried about online scams."

Chatbot: "It's completely understandable to feel that way. Scammers can be very convincing. Let me share some tips to help you stay safe."

- And it must then share a tip and does not make the user enter again for something on the topic

7. Error Handling and Edge Cases

Objective: Ensure the chatbot continues to function smoothly when it encounters unexpected inputs or unrecognised keywords.

Tasks

- Create a default response for unknown inputs, such as "I'm not sure I understand. Can you try rephrasing?"
- Maintain functionality without crashes or terminations due to unexpected inputs, which helps build a professional-grade chatbot.

8. Code Optimisation

Objective: Encourage the use of efficient data structures and Object-Oriented Programming (OOP) practices to manage responses.

Tasks

- Use dictionaries, lists, or arrays to organise keyword responses and random selections, optimising code readability and performance.
- Ensure the use of **methods** and **classes** and good organisation practices.
- Ensure the code is ready for further expansion in Part 3/POE.

Submission Instructions

ARC Submission

- Submit **ONLY** your GitHub link on ARC, which includes the following:
  - Complete project folder, including source code, the README file, and all multimedia files (WAV file for the voice greeting and ASCII art image).
- Submit your presentation for Task 2 as a **YouTube unlisted link**. Ensure the video is clearly recorded with a voice-over, explaining the code structure, logic, and techniques used.

GitHub Submission

- Ensure your repository has a **minimum of six commits** with meaningful messages.
- Ensure that there is a tag with a minimum of two releases.

Part 3/POE — Cybersecurity Awareness Chatbot (Marks: 100)

All learning units.

- Enhance the existing GUI from Task 2 with additional functionalities.
- Use controls to create a graphical user interface.

Description

In this final part, you will complete your Cybersecurity Awareness Chatbot by adding advanced features that make it more engaging and educational. You'll introduce a:

1. **Task assistant** to help users manage cybersecurity-related tasks.
2. **Mini game** that quizzes users on cybersecurity knowledge.
3. **Natural Language Processing (NLP) simulation** to make the chatbot more responsive to user inputs.
4. **Activity Log Feature,** which will record the actions that the bot has taken

Note: Part 3 is solely a **GUI-based application**. If a student submits a console application instead of a GUI, they will receive a score of zero for this part. The GUI must be implemented using XAML / Windows Forms with appropriate controls, graphics rendering, and styles.

Questions

Task 1: Task Assistant with Reminders (GUI)

Objective: Create a task assistant feature where users can manage cybersecurity-related tasks, such as "Enable two-factor authentication". This feature will help users stay organised with their cybersecurity efforts.

What You Need to Do

Continue from Your Part 1 and Part 2 Code

- This feature must be integrated with the existing chatbot structure.
- Users should still be able to access features from Part 1 and Part 2, such as dynamic responses, keyword recognition, and sentiment detection.

Database Integration (Task Storage)

In this **Task assistant,** you must integrate a simple database (**MySQL**) to store the user's cybersecurity tasks.

Add a Task with Details and a Reminder in the DB

- Allow users to add tasks related to cybersecurity, like "*Set up two-factor authentication*" or "*Review account privacy settings.*"
- Each task should include a **title**, **description**, and an **optional reminder** (e.g., "*Remind me in 7 days*") and should be stored in the DB.
- Users should be able to **specify a date** or **timeframe** for the reminder.

View and Manage Tasks

- Display all tasks to the user, showing the title, description, and any reminders they've set.
- Provide options to delete a task or mark it as completed; these changes should also reflect in the DB.
- Ensure that the chatbot's responses remain consistent with the previous parts when managing these tasks.

Example of Interaction

User: Add task - Review privacy settings

Chatbot: Task added with the description "Review account privacy settings to ensure your data is protected." Would you like a reminder?

User: Yes, remind me in 3 days.

Chatbot: Got it! I'll remind you in 3 days.

Task 2: Cybersecurity Mini-Game (Quiz) - GUI

Objective: Integrate a quiz game that tests the user's knowledge of cybersecurity. This game will reinforce learning and add an interactive element to the chatbot. **You can continue using lists/dictionaries for this Quiz.**

What You Need to Do

Build on Your Existing Project

- Add this mini game as an additional feature within the same chatbot project.
- Users should be able to access this mini game along with the other features from Parts 1 and 2.

Prepare more than 10 Cybersecurity Questions

- The questions should cover basic cybersecurity topics, such as phishing, password safety, safe browsing, and social engineering.
- **Mix question types** with multiple-choice and true/false formats for variety.
- Questions should show one at a time.

Provide Immediate Feedback

- After each answer, tell the user if they got it right or wrong and provide a brief explanation.
- Reinforce important cybersecurity concepts through this feedback.

Track the User's Score

- Keep track of correct answers and display a final score at the end of the game.
- Based on the score, give feedback, such as "*Great job! You're a cybersecurity pro!*" or "*Keep learning to stay safe online!*"

Example Question

Question: What should you do if you receive an email asking for your password?
A) Reply with your password
B) Delete the email
C) Report the email as phishing
D) Ignore it
Chatbot: Correct! Reporting phishing emails helps prevent scams.

Task 3: Natural Language Processing (NLP) Simulation (GUI)

Objective: Continue using the list/dictionaries like before and make the chatbot more responsive by recognising different ways users might phrase requests or questions.

Your chatbot needs to recognise user requests and commands even if they are worded differently. For instance, users might ask to "add a task" or "set a reminder" in various ways, and the chatbot should respond appropriately.

What You Need to Do

Build on Existing Features

- Integrate NLP functionality with the current chatbot setup. For instance, if a user says, "*Can you remind me to update my password?*" the chatbot should recognise this as a task to add a reminder, thanks to keyword detection.

Use Keyword Detection

- Identify keywords in the user's input related to cybersecurity, tasks, or quiz questions, such as "task", "quiz", "reminder", "password", or "phishing".
- Make the chatbot flexible enough to understand user requests even if they're worded differently (e.g., "Add a reminder to check my privacy settings").
- "I didn't quite understand that. Could you rephrase?" Should be limited as much as possible.

Simulate NLP with Basic String Manipulation

- Use simple string manipulation methods (such as string.Contains() in C#) to detect keywords and understand the user's intent.
- **Allow for slight variations** in user input. For example, if a user types "Add a task to enable 2FA," the bot should recognise "Add task" and "2FA" as keywords to create a new task.

Research Basic NLP Techniques

- **Note**: *This part will require you to do some research, as your lecturer may or may not provide further guidance.*
- Look up simple NLP methods in C# or experiment with regular expressions to create a more sophisticated keyword-matching system.

Example of the Interaction

User: "Remind me to update my password tomorrow."

Chatbot: "Reminder set for 'Update my password' on tomorrow's date."

User: "Add a task to enable two-factor authentication."

Chatbot: "Task added: 'Enable two-factor authentication.' Would you like to set a reminder for this task?"

User: "What have you done for me?"

Chatbot: "Here's a summary of recent actions:

1. Reminder set for 'Update my password' tomorrow.
2. Task added: 'Enable two-factor authentication' (no reminder set).

Task 4: Activity Log Feature (GUI)

Objective: Record a log of actions that the chatbot has taken and provide a way for the user to view this log upon request. This log will include details like tasks added, reminders set, quiz attempts, and other key actions. The **user** should be able to access this log by providing a command to the bot.

What You Need to Do

Create an Activity Log

- **Add a list or dictionary in your code** to store the actions the chatbot has completed, including **timestamps** or short descriptions if possible.
- For every significant action (e.g., adding a task, setting a reminder, starting the quiz), add a short description of the action to the log.

Track Key Actions

Log actions such as

- Tasks: When a task is added, updated, or marked as completed.
- Reminders: When a reminder is set for a task.

- Quiz Activity: When the user starts or completes the quiz.
- NLP Interactions: Any actions taken based on NLP interpretations, such as custom responses or commands recognised through keyword detection.
- Allow users to type a command like “Show activity log” or “What have you done for me?” to view the log.
- When this command is triggered, display the log of recent actions in a list format, with brief descriptions of each action.
- **Display only the last 5–10 actions** to keep the log concise and relevant. Optionally, you can add a “Show more” feature if you wish to show the full history.

Example of Activity Log

User: "Show activity log."

Chatbot:

"Here’s a summary of recent actions:

1. Task added: 'Enable two-factor authentication' (Reminder set for 5 days from now).
2. Quiz started - 5 questions answered.
3. Reminder set: 'Review privacy settings' on [specific date]."

Submission Instructions

ARC Submission

- Submit **ONLY** your GitHub link on ARC, which includes the following:
  - Complete project folder, including source code, the README file, and all necessary files.
- Submit your presentation for Task 3/POE as a **YouTube unlisted link**. Ensure the video is clearly recorded with a voice-over, explaining the code structure, logic, and techniques used.

GitHub Submission

- Ensure your repository has a **minimum of six commits** with meaningful messages.
- Ensure that there is a tag with a minimum of three releases.

Assessment Sheet (Marking Rubric)

Please note: Tear off this section and **attach** it to your work when you submit it/ If this is an online submission, then this information needs to be included in the online submission.

MODULE NAME:	**MODULE CODE:**
PROGRAMMING 2A	**PROG6221/w**

STUDENT NAME:
STUDENT NUMBER:

PART 1					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Correct Submission [5 Marks]	• Missing required files. • No README or instructions.	• Contains all required files. • README with basic info.	• Organised and well-structured. • README covers usage and setup.	• Clear folder structure, includes README. • README includes detailed usage/setup.	
	0 – 1 Mark	**2 - 3 Marks**	**4 Marks**	**5 Marks**	
GitHub and CI Setup **[15 Marks]**	• No commits made to track changes.	• Commit messages lack detail and clarity.	• Includes a good commit history showing progress. • GitHub Actions are set up for Continuous Integration (CI) but may have minor issues.	• Well-documented six commit messages that describe changes clearly. • GitHub Actions are fully functional, successfully running checks or tests on each commit.	
	0 – 5 Marks	**6 – 10 Marks**	**11 – 13 Marks**	**14 – 15 Marks**	

PART 1					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Voice Greeting and Image Display **[10 Marks]**	• No voice greeting or image is present in the project. • Media files are incomplete or missing.	• A basic voice greeting or ASCII art is included. • Implementation is minimal and lacks engagement.	• Both a clear voice greeting and engaging ASCII art are included. • The media elements work together to create a more immersive experience.	• Creative and visually appealing ASCII art, along with high-quality audio. • Media elements enhance the user experience significantly, making the chatbot feel interactive.	
	0 – 2 Marks	**3 - 5 Marks**	**6 - 8 Marks**	**9 - 10 Marks**	
Greeting and User Interaction **[10 Marks]**	• No personalised greeting provided to users. • User interaction feels robotic and disengaging.	• Basic greeting implemented but lacks personalisation. • Limited acknowledgement of user input (e.g., name).	• Personalised greetings that engage users and ask for their names. • The chatbot responds in a clear and friendly manner.	• Text greeting after ASCII and audio. • Highly professional and engaging conversation style. • The chatbot consistently personalises responses, creating an inviting atmosphere.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	
Basic Response System **[15 Marks]**	• Fails to provide key responses to user queries. • Responses are generic and unhelpful.	• The chatbot responds to 2+ basic queries, but the responses lack depth.	• At least 3 specific and relevant responses are implemented. • Responses show an understanding of key topics.	• Responses cover a wide range of questions with depth and clarity. • The conversation feels natural and varied, keeping users engaged.	
	0 – 4 Marks	**5 - 8 Marks**	**9 – 12 Marks**	**13 - 15 Marks**	

PART 1					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Input Validation **[5 Marks]**	• No input validation implemented, leading to crashes or errors. • The chatbot does not handle invalid input gracefully.	• Basic validation checks are in place, but they are minimal. • General error handling may not cover all edge cases.	• Comprehensive validation that effectively handles most user inputs. • Default responses provided for unknown inputs.	• Exception handling is graceful and informative, guiding users with helpful messages when errors occur.	
	0 – 1 Mark	**2 - 3 Marks**	**4 Marks**	**5 Marks**	
Enhanced Console UI **[10 Marks]**	• Text output is unstructured and difficult to read. • No visual enhancements or formatting present.	• Minimal styling applied to console output; some basic formatting used.	• Colour coding and structure are evident, enhancing readability. • Visual elements improve user experience but may be inconsistent.	• Polished, well-formatted console UI that is visually appealing. • Effective use of colour, borders, and spacing creates an engaging interface.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	
Code Structure and Readability **[15 Marks]**	• Code is poorly organised, making it hard to follow. • Lacks basic structure and consistency.	• Code structure is basic but functional; some effort is made at organisation. • Minimal comments provided.	• Code is modular and organised with consistent naming conventions. • Some comments provide clarity on function purpose.	• Professional structure with good use of methods and classes, and documentation. • Full comments/ documentation enhance understanding and maintainability.	
	0 – 4 Marks	**5 - 8 Marks**	**9 – 12 Marks**	**13 - 15 Marks**	

PART 1					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Video Presentation [15 Marks]	• No video presentation provided • presentation lacks clarity or preparation. • Limited understanding of project features or functionality.	• Video has basic presentation of key features and functionality, with limited explanation.	• Clear and well-prepared video presentation. • Good understanding of project goals and features. • Code is demonstrated with clarity and engagement.	• Engaging, professional presentation with strong explanations. • Excellent understanding of project concepts and goals. • Code is thoroughly demonstrated, covering key aspects and functionality.	
	0 – 4 Marks	**5 - 8 Marks**	**9 – 12 Marks**	**13 - 15 Marks**	
No GitHub used [-5%]	No GitHub used				
	-5%				

[TOTAL MARKS: 100]

MODULE NAME:	MODULE CODE:
PROGRAMMING 2A	PROG6221/w

STUDENT NAME:
STUDENT NUMBER:

PART 2					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Correct Submission **[5 Marks]**	• Missing required files. • No README or instructions.	• Contains all required files. • README with basic info.	• Organised and well-structured. • README covers usage and setup.	• Clear and logical folder structure with all necessary files. • GitHub link submitted on ARC • YouTube video presentation link on ARC or in README • README includes detailed usage instructions, examples, and setup steps.	
	0 – 1 Mark	**2 - 3 Marks**	**4 Marks**	**5 Marks**	
GitHub and Releases **[10 Marks]**	• No commits made to track changes or tags/releases missing.	• Repository created with basic commits and tags/releases. • Commit messages lack detail and clarity.	• Good commit history with structured tags/releases. • Release notes included but may be brief.	• Well-documented six commit messages describing changes clearly. • A release with minimum of 3 tags each with clear version notes.	
	0 – 2 Marks	**3 – 5 Marks**	**6 – 8 Marks**	**9 – 10 Marks**	
Keyword Recognition **[15 Marks]**	• Lacks keyword recognition or specific responses. • Static, unresponsive interaction.	• Recognises basic keywords with limited responses. • Interaction lacks depth.	• Recognises at least 3 specific keywords with targeted responses. • Effective and relevant responses provided.	• Responds naturally to a range of keywords, providing varied, engaging responses.	
	0 – 4 Marks	**5 - 8 Marks**	**9 - 12 Marks**	**13 - 15 Marks**	

PART 2					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Random Responses **[10 Marks]**	• No random responses implemented. • Responses are static or repetitive.	• Basic random response system included, with limited variety.	• Responses vary meaningfully for multiple questions. • Adds interest to interactions.	• Well-developed random responses, creating a natural, varied conversation.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	
Conversation Flow **[10 Marks]**	• No continuity in conversation; resets frequently. • Lacks logical progression.	• Basic flow maintained, but limited continuity.	• Smooth handling of follow-up questions, maintaining natural flow.	• Seamless conversational flow, engaging and logically progressing.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	
Memory and Recall **[10 Marks]**	• No memory functionality; cannot recall previous inputs.	• Limited memory; recalls basic information inconsistently.	• Remembers multiple user inputs effectively, adding personalisation.	• Consistent, responsive memory, with seamless recall that enhances engagement.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	
Sentiment Detection **[10 Marks]**	• Lacks sentiment detection; tone remains static.	• Basic sentiment detection for 1-2 emotions; limited response variation.	• Effective sentiment detection with responses that adapt appropriately.	• Advanced sentiment detection, with dynamic responses tailored to mood/tone.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	
Code Structure and Optimisation **[10 Marks]**	• Code is poorly structured and disorganised. • Lacks comments and readability.	• Basic code structure with some comments, but organisation needs improvement.	• Code is modular and organised, with clear naming conventions. • Code is reasonably well-commented.	• Professional structure, optimised for clarity and maintainability. • Good use of classes and methods. • Full comments/ documentation enhance understanding.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	

PART 2					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Chat Bot GUI Design **[10 Marks]**	• GUI is missing or unusable. • Layout is cluttered or unclear. • Input/output areas not functioning.	• Basic GUI with input and output area. • User can type and receive a response. • Functional but plain design.	• Clean, structured GUI with organised sections. • Responses display clearly with spacing. • User interaction is smooth.	• Highly polished, visually appealing interface. • Professional layout, colours, ASCII art and Voice greeting. • Very smooth flow and intuitive user experience.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	
Video Presentation **[10 Marks]**	• No video presentation provided • presentation lacks clarity or preparation. • Limited understanding of project features or functionality.	• Video has basic presentation of key features and functionality, with limited explanation.	• Clear and well-prepared video presentation. • Good understanding of project goals and features. • Code is demonstrated with clarity and engagement.	• Engaging, professional presentation with strong explanations. • Excellent understanding of project concepts and goals. • Code is thoroughly demonstrated, covering key aspects and functionality.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	
No GitHub used [-5%]	No GitHub used				
	-5%				

[TOTAL MARKS: 100]

MODULE NAME:	MODULE CODE:
PROGRAMMING 2A	PROG6221/w

STUDENT NAME:
STUDENT NUMBER:

POE					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Correct Submission [5 Marks]	• Missing required files. • No README or instructions.	• Contains all required files. • README with basic info.	• Organised and well-structured. • README covers usage and setup.	• Clear and logical folder structure with all necessary files. • GitHub link submitted on ARC • YouTube video presentation link on ARC/README • README includes detailed usage instructions, examples, and setup steps.	
	0 – 1 Mark	**2 - 3 Marks**	**4 Marks**	**5 Marks**	
GitHub and Releases with Tags [10 Marks]	• No commits, tags, or releases included.	• Repository created with basic commits, tags/releases. • Commit messages lack detail and clarity.	• Includes good commit history with detailed tags/releases and release notes.	• Well-documented six commit messages • A release with minimum of 3 tags each with clear version notes.	
	0 – 2 Marks	**3 – 5 Marks**	**6 – 8 Marks**	**9 – 10 Marks**	
Task Assistant with Reminders (GUI Implementation) [15 Marks]	• Task assistant feature is missing or doesn't function. • No reminders included.	• Basic task assistant included, but with limited features. • Simple reminder system.	• Fully functional task assistant with descriptions and reminders.	• Task assistant is intuitive and user-friendly, with a robust reminder system.	
	0 – 4 Marks	**5 - 8 Marks**	**9 - 12 Marks**	**13 - 15 Marks**	

POE					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Task Assistant Database Integration **[15 Marks]**	• No working DB. Tasks are not stored or loaded from DB.	• Basic DB connection. Tasks can be saved OR loaded, but not fully consistent.	• Tasks are saved and loaded consistently. Most actions reflect correctly in DB.	• Robust, error-handled DB integration. All CRUD actions (add, read, mark as complete, delete) sync correctly between GUI and DB.	
	0 – 4 Marks	**5 - 8 Marks**	**9 – 12 Marks**	**13 -15 Marks**	
Cybersecurity Mini-Game (Quiz) with GUI **[15 Marks]**	• Mini-game is missing or lacks basic functionality. • Few or no interactive questions.	• Basic quiz with limited feedback; minimal variety in questions.	• Engaging quiz with 10 questions, covering key cybersecurity topics. • Provides feedback for each answer.	• 10+ questions • Interactive, engaging quiz that covers cybersecurity thoroughly, with smooth, **varied** feedback and flow.	
	0 – 4 Marks	**5 - 8 Marks**	**9 – 12 Marks**	**13 -15 Marks**	
NLP Simulation with GUI Interaction **[10 Marks]**	• No NLP functionality implemented. • Lacks keyword detection and phrase handling.	• Basic keyword detection, but limited in variation or adaptability.	• Flexible detection of different phrases, responding naturally to user inputs	• Advanced NLP simulation that adapts to user phrasing, creating an engaging interaction.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	
Activity Log Feature with GUI **[10 Marks]**	• No activity log feature implemented. • Cannot view past actions.	• Basic activity log included but lacks detail or clarity. • Lacks limiting Log Activity Length	• Log provides clear summaries of key actions, enhancing user experience.	• Comprehensive activity log, with organised, clear summaries that are easy to navigate. • Shows 5-10 at a time, allows show more option.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	
Combining Parts 1, 2, and 3 **[10 Marks]**	• Parts 1 and 2 are not integrated; functionality is disjointed.	• Basic integration of features, with limited cohesion.	• Smooth integration with consistent functionality across parts.	• Cohesive integration, providing a seamless, professional user experience across all features.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	

POE					
Marking Criteria	**Does not meet the required standard**	**Meets the required standard**	**Partially exceeds the required standard**	**Greatly exceeds the required standard**	**Feedback**
Video Presentation **[10 Marks]**	• Video lacks clarity, is unprepared, or does not explain the chatbot's functionality well. • Minimal or confusing explanations of key features.	• Covers basic functionality but lacks depth in explanations. • Limited engagement with the audience, missing some technical details.	• Clear, well-organised video presentation. • Demonstrates a good understanding of the chatbot's functionality and code. • Engages the viewer, providing detailed explanations for key features.	• Professional, engaging video presentation. • Thoroughly explains all key features, including detailed explanations of the code and logic behind each part. • Demonstrates an in-depth understanding of the chatbot's purpose and functionality, making complex concepts easy to understand.	
	0 – 2 Marks	**3 - 5 Marks**	**6 – 8 Marks**	**9 -10 Marks**	
No GitHub used [-5%]	No GitHub used				
	-5%				

[TOTAL MARKS: 100]